cloud-messenger™

BUILD
DEPLOY
LEARN
GROW

PRODUCTION HANDBOOK · EDITION 1.0

Deploy a Containerized App on AWS

Build Today for a Bigger Tomorrow

From a clean Ubuntu box to a live Fargate container. Docker, AWS CLI, Amazon ECR and Amazon ECS — explained, verified and hardened for production.

```
$ code
$ build
$ deploy
$ repeat|
```

SAME SKILLS. BIGGER POSSIBILITIES.

01 Ubuntu Setup

02 Install Docker

03 AWS CLI Configure

04 Amazon ECR Repository

05 ECS Cluster & IAM

06 Task, Run & Harden

CREATED BY

cloud-messenger™

cloudmessenger.in · #buildwithCloud-Messenger

CLOUD SKILLS · REAL IMPACT

© 2026 cloudmessenger. All rights reserved.

START HERE

Welcome to the Handbook

Everything you need to ship a containerized Node.js app to AWS — and keep it healthy.

THE LAB AT A GLANCE

APP	Node.js To-Do (node-todo-cicd)
CONTAINER PORT	8000 / TCP
COMPUTE	AWS Fargate (serverless)
REGISTRY	Amazon ECR
REGION	e.g. ap-south-1 (Mumbai)
TIME TO FINISH	≈ 45-60 minutes

HOW TO USE THIS HANDBOOK

- i — Pro tip — a shortcut or best practice.
- ! — Watch out — a common trap in this lab.
- ! — Security — protect keys, ports and data.
- P — Production — what to change before real users.

Each phase follows the same rhythm: read the goal, run the numbered steps, then confirm with the “verify” check before moving on. Replace anything in <angle-brackets> with your own value. Commands assume Ubuntu 22.04/24.04 on x86_64.

CONTENTS

00 **Architecture & Prerequisites** — The big picture and what to prepare — 3

01 **Ubuntu & Code Setup** — Update, clone, inspect the Dockerfile — 4

02 **Docker: Build & Run** — Install, permissions, local test — 5

03 **AWS CLI & Credentials** — Install v2, configure, verify identity — 6

04 **Amazon ECR** — Create the repository, tag and push — 7

05 **ECS Cluster & IAM Roles** — Fargate cluster, execution role — 8

06 **Task Definition** — CPU, memory, container, port 8000 — 9

07 **Run, Secure & Access** — Run task, security group, logs — 10

08 **Troubleshooting Matrix** — Symptom → cause → fix — 11

09 **Production Hardening** — Lab practice vs production practice — 12

10 **Cleanup & Cheat Sheet** — Stop the bill; one-page recap — 13

A **Appendix: Original Visual Guides** — Whiteboards, infographics, notes reconciliation — 14

ONE SOURCE OF TRUTH

This handbook merges your whiteboard notes, hand-written steps and the Cloud-Messenger visual guides into one flow. Where those sources disagreed (public vs private repo, ARM vs x86, ports), the differences are reconciled on page 15.

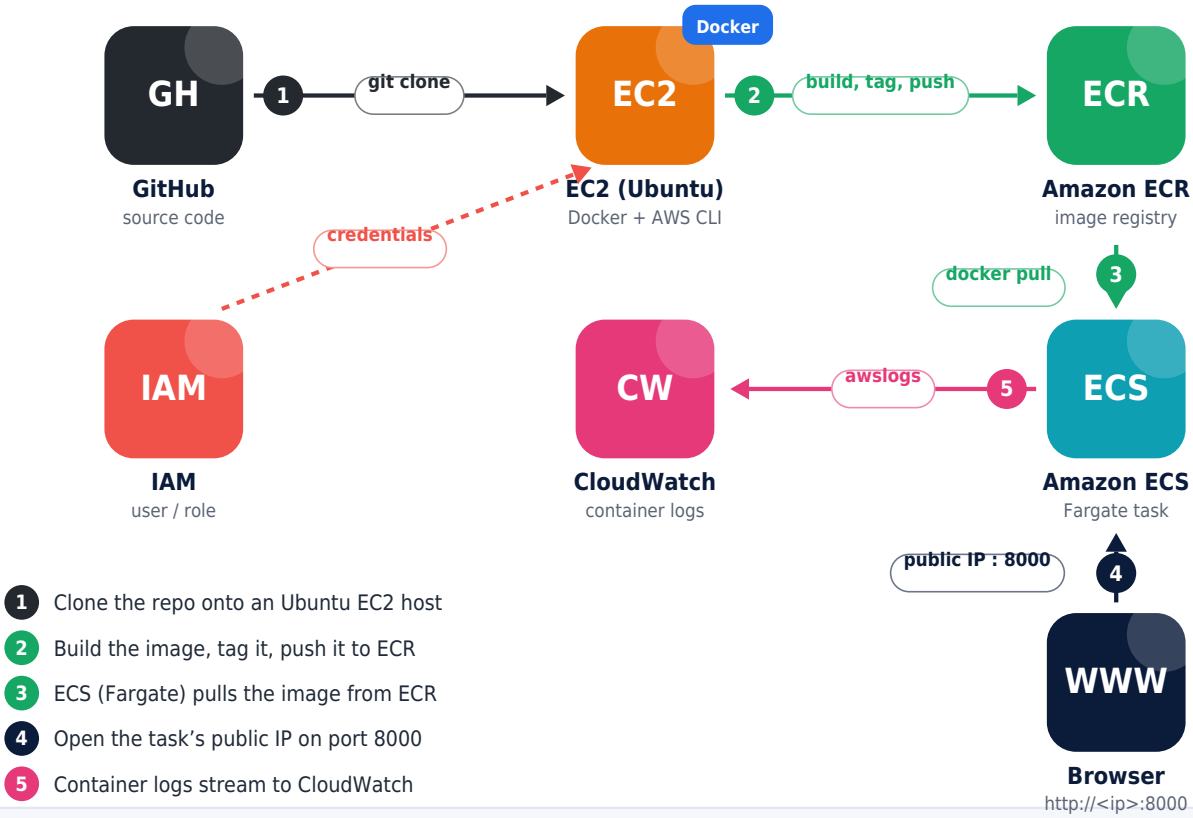
© 2026 cloudmessenger. All rights reserved.

00

THE BIG PICTURE

Architecture & Prerequisites

How code travels from GitHub to a running Fargate container.



1. Clone the repo onto an Ubuntu EC2 host
2. Build the image, tag it, push it to ECR
3. ECS (Fargate) pulls the image from ECR
4. Open the task's public IP on port 8000
5. Container logs stream to CloudWatch

PREREQUISITES CHECKLIST

- ☑ **AWS account** — with billing alerts enabled
- ☑ **IAM user (not root)** — with ECR + ECS permissions
- ☑ **EC2 Ubuntu instance** — t3.micro is enough; key pair saved
- ☑ **EC2 security group** — SSH 22 from your IP; 80 for local test
- ☑ **GitHub repo** — node-todo-cicd with a Dockerfile
- ☑ **Default VPC + public subnet** — for the Fargate task
- ☑ **Region chosen** — e.g. ap-south-1; keep it consistent
- ☑ **A terminal** — ssh, or EC2 Instance Connect

DEPENDENCY CHAIN

Why this order? Docker and the CLI must exist before you can push; the image must exist in ECR before a task can pull it; the execution role must exist before the task can start. Each phase unlocks the next.

© 2026 cloudmessenger. All rights reserved.

01

PHASE 1 · EC2 & UBUNTU

Ubuntu & Code Setup

Get a clean server, update it, and pull down the project.

1 Launch & connect to Ubuntu EC2

Launch an Ubuntu instance with your key pair. Open SSH (22) from your IP only, then connect.

```
ssh -i <key>.pem ubuntu@<ec2-public-ip>
```

2 Update system packages

Refresh the package index so installs pull current versions.

```
sudo apt update -y
```

3 Clone the repository

Download the project code from GitHub.

```
git clone https://github.com/
 Cloud-Messenger/node-todo-cicd.git
```

4 List files

Confirm the repository folder was created.

```
ls
```

5 Enter the project folder

Everything that follows is run from inside this directory.

```
cd node-todo-cicd
```

6 Read the Dockerfile

See exactly how the image will be built and which port it exposes.

```
ls
cat Dockerfile
```

REFERENCE DOCKERFILE

Dockerfile

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8000
CMD ["node", "server.js"]
```

READ IT FIRST

Your repo's own Dockerfile is the source of truth. What matters: the app listens on 0.0.0.0:8000 and the file declares EXPOSE 8000.

UPGRADE THE BASE

node:18 has reached end-of-life. For real workloads use an LTS tag such as node:22-slim and run as a non-root user.

CHECKPOINT

Checkpoint: ls shows Dockerfile and package.json, and the Dockerfile exposes port 8000. If the clone failed, check the URL and that git is installed (sudo apt install git).

PRO TIP

Some guides start with "sudo su". It works for a lab, but prefer prefixing single commands with sudo so you never run a whole session as root.

© 2026 cloudmessenger. All rights reserved.

02

PHASE 2 · DOCKER

Docker: Build & Run

Install Docker, build the image, and prove it works locally before touching AWS.

1 Install Docker Engine

The docker.io package ships the engine and CLI on Ubuntu.

```
sudo apt install docker.io -y
docker --version
```

2 Run Docker without sudo

If “docker ps” says permission denied, add your user to the docker group, then reboot or log in again.

```
docker ps
whoami
sudo usermod -aG docker $USER
sudo reboot
```

3 Build the image

Builds an image named nodeapp from the Dockerfile in the current folder (note the dot).

```
docker build -t nodeapp .
docker images
```

4 Run the container

Detached (-d), named, host port 80 mapped to container port 8000.

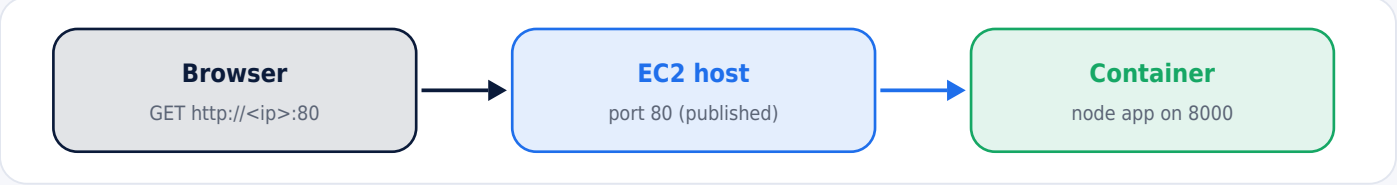
```
docker run -d --name nodeappweb -p 80:8000 nodeapp
docker ps
```

5 Test in the browser

Open the EC2 public IP (security group must allow port 80), or test from the server itself.

```
curl -I http://localhost
# browser: http://<ec2-public-ip>
```

HOW -p 80:8000 WORKS



daily commands

```
docker logs nodeappweb          # view output
docker stop nodeappweb          # stop it
docker rm nodeappweb            # remove it
docker run -d -p 8080:8000 nodeapp   # if 80 is busy
```

TYPO ALERT

Handwritten notes say “usermode” and “Reboot”. The real commands are lowercase: usermod and reboot.

CHECKPOINT

Verify: docker ps shows nodeappweb as Up, and the To-Do page loads. Only then move on.

© 2026 cloudmessenger. All rights reserved.

PHASE 3 · AWS CLI

03 AWS CLI & Credentials

Install CLI v2 and prove who you are to AWS.

1 Install helper utilities

unzip and curl are needed to fetch and extract the installer.

```
sudo apt install -y unzip curl
```

2 Remove any old CLI

The apt awscli package is v1 — remove it to avoid version clashes.

```
sudo apt remove awscli -y
```

3 Download the v2 installer

x86_64 shown. On Graviton/ARM hosts use awscli-exe-linux-aarch64.zip.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

4 Unzip and install

Extract, then run the bundled installer. Confirm the version.

```
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

CONFIGURE

aws configure

```
aws configure
> AWS Access Key ID [None]: <your-key-id>
> AWS Secret Access Key [None]: <your-secret>
> Default region name [None]: ap-south-1
> Default output format [None]: json
```

verify

```
aws sts get-caller-identity
aws ecr describe-repositories --region ap-south-1
```

Where do I get the keys?

1. Console → IAM → Users
2. Select your IAM user
3. Security credentials tab
4. Create access key → copy both

CHECKPOINT

A JSON reply with your Account and Arn means the CLI works.

IAM PERMISSIONS FOR THE CLI USER

Managed policy	Registry	Why
AmazonElasticContainerRegistryPublicFullAccess	Public	Create repos and push images (lab)
AmazonElasticContainerRegistryPublicPowerUser	Public	Push/pull without repo deletion
AmazonElasticContainerRegistryPublicReadOnly	Public	Pull and list only
AmazonEC2ContainerRegistryPowerUser	Private	Push/pull for private ECR (production)

PROTECT YOUR KEYS

Never paste access keys into code, screenshots or Git. Better still: attach an IAM role to the EC2 instance and skip “aws configure” entirely — the CLI then uses short-lived credentials automatically.

© 2026 cloudmessenger. All rights reserved.

PHASE 4 · AMAZON ECR

04 Amazon ECR Repository

Create the registry that ECS will pull your image from, then push to it.

1 Open Amazon ECR

In the console search bar type “ECR” and open Elastic Container Registry.

2 Create the repository

Public registry → Repositories → Create repository. Name it node-app, set Linux and pick the architecture that matches your build host (x86-64 for a t3 instance).

3 Open “View push commands”

Click the repo, then View push commands. The console prints the exact four commands with your alias filled in.

PUSH TO PUBLIC ECR (run on the EC2 host, inside the project folder)

public ECR

```
# 1. Authenticate Docker to public ECR
aws ecr-public get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin public.ecr.aws
# 2. Build
docker build -t node-app .
# 3. Tag
docker tag node-app:latest public.ecr.aws/<alias>/node-app:latest
# 4. Push
docker push public.ecr.aws/<alias>/node-app:latest
```

PRODUCTION VARIANT: PRIVATE ECR

private ECR

```
aws ecr get-login-password --region <region> | \
  docker login --username AWS --password-stdin \
  <account-id>.dkr.ecr.<region>.amazonaws.com
docker tag nodeapp:latest \
  <account-id>.dkr.ecr.<region>.amazonaws.com/nodeapp:latest
docker push <account-id>.dkr.ecr.<region>.amazonaws.com/nodeapp:latest
```

REGION QUIRK

Public ECR logins always use us-east-1, whatever region you work in.

KEEP THIS

Copy the Image URI from the repo page — you paste it into Phase 6.

	Public ECR (lab)	Private ECR (production)
Pull access	Anyone, no auth needed	Only IAM principals you allow
Good for	Learning, open-source images	Company apps, anything proprietary
Extras	Basic	Scan on push, immutable tags, lifecycle rules

© 2026 cloudmessenger. All rights reserved.

05

PHASE 5 · AMAZON ECS

ECS Cluster & IAM Roles

Create the Fargate cluster and the role that lets ECS pull images and write logs.

1 Open ECS and create a cluster
Amazon ECS → Clusters → Create cluster.

2 Name it
Use something recognisable, e.g. <your-name>-cluster.

3 Choose infrastructure
AWS Fargate (serverless) only — no EC2 nodes to patch or scale.

4 Enable monitoring
Turn on Container Insights for CPU, memory and network metrics.

5 Create
Wait for the cluster status to show Active.

CREATE THE TASK EXECUTION ROLE

IAM → Roles → Create role → trusted entity: AWS service → use case: Elastic Container Service Task. Name it ECSExeRole (or ecsTaskExecutionRole) and attach policies from the table. The execution role is what ECS itself uses to pull the image and ship logs.

Managed policy (from original lab)	Needed?	What it does
AmazonECSTaskExecutionRolePolicy	REQUIRED	Pull from private ECR, write to CloudWatch Logs
AmazonElasticContainerRegistryPublicReadOnly	Lab	Pull from public ECR (public pulls work without it)
AmazonElasticContainerRegistryPublicPowerUser	Optional	Push/pull public ECR — belongs to a human, not a task
AmazonElasticContainerRegistryPublicFullAccess	Optional	Full public ECR admin — too broad for a task
EC2InstanceProfileForImageBuilder	Not needed	Image Builder pipelines, unrelated to this flow
EC2InstanceProfileForImageBuilderECRContainerBuilds	Not needed	Image Builder container builds

LEARN FROM THE LIST
Least privilege: for this lab the single policy AmazonECSTaskExecutionRolePolicy is enough. The extra policies came from experimenting.

SERVICE-LINKED ROLE
If no ECS role exists, AWS creates the service-linked role AWSServiceRoleForECS automatically the first time you create a cluster.

© 2026 cloudmessenger. All rights reserved.

06

PHASE 6 · TASK DEFINITION

Task Definition

The blueprint ECS uses to start your container: image, size, port, roles, logs.

ANATOMY

TASK DEFINITION · family: nodeapp-task · Fargate · Linux/X86_64

CONTAINER: nodeapp-container

```
image: public.ecr.aws/<alias>/node-app:latest
port mapping: 8000 / TCP    essential: yes
logs: awslogs → /ecs/nodeapp-task
```

Execution role: ECSExeRole

Task size: vCPU + memory

Network mode: awsvpc

1 Create the task definition

ECS → Task definitions → Create new task definition. Enter a family name and choose AWS Fargate as launch type.

2 Set platform and size

OS: Linux. Architecture must match the image you pushed. Start small; the original lab used 1 vCPU / 3 GB.

3 Attach roles

Task execution role: ECSExeRole (Phase 5). Task role: only if your app calls AWS APIs.

4 Add the container

Name it, paste the ECR Image URI, add port mapping 8000 / TCP. Leave resource limits blank or ≤ task size.

5 Create

Review and Create. You get a revision such as nodeapp-task:1.

Setting	Lab value	Guidance
Task size	0.25 vCPU / 0.5 GB	Enough for a To-Do app. Original notes used 1 vCPU / 3 GB — over-provisioned.
Architecture	Linux / X86_64	Must equal the CPU architecture of the machine that built the image.
Container port	8000 / TCP	Same as EXPOSE in the Dockerfile and your app's listen port.
Log collection	awslogs ON	Creates /ecs/<family> in CloudWatch. Your first debugging tool.

! **LIMITS TRAP**

Whiteboard notes list container CPU 2 / memory 8 GB while the task is 1 vCPU / 3 GB. Container limits can never exceed the task total — a mismatch makes registration fail. When unsure, leave container limits empty.

© 2026 cloudmessenger. All rights reserved.

07

PHASE 7 · RUN & ACCESS

Run, Secure & Access

Start the task, open the right port, and reach your app on its public IP.

1 Run new task
Cluster → Tasks → Run new task. Compute: Launch type Fargate. Application type: Task.

2 Pick the blueprint
Family: nodeapp-task, Revision: latest, Desired tasks: 1.

3 Configure networking
VPC: default. Subnet: a public one. Public IP: ENABLED. Security group: next step.

4 Open port 8000
Create or edit the security group so the task accepts traffic on the app port.

5 Create and wait
Watch the status move to RUNNING (1–2 minutes).

6 Open the app
Task details → Networking → copy Public IP → browse to http://<public-ip>:8000

SECURITY GROUP · INBOUND

Type	Proto	Port	Source
Custom TCP	TCP	8000	0.0.0.0/0 (lab)
Custom TCP	TCP	8000	<your-ip>/32 (better)

TASK LIFECYCLE

PROVISIONING → PENDING → ACTIVATING → RUNNING

verify from the CLI

```
aws ecs list-tasks --cluster <cluster>
aws ecs describe-tasks --cluster <cluster> \
  --tasks <task-arn> \
  --query 'tasks[0].[lastStatus,stoppedReason]'
aws logs tail /ecs/nodeapp-task --follow
```

PORT 80?
The original guide opens 80 and 8000. The task only listens on 8000, so 80 is unnecessary here.

ROLE OVERRIDE
If the task will not start, set the execution role under Task overrides.

CHECKPOINT
Page loads? You are done — your container runs on Fargate, pulled from ECR, logging to CloudWatch. If not, go straight to the Troubleshooting Matrix.

© 2026 cloudmessenger. All rights reserved.
#buildwithCloud-Messenger

08

TROUBLESHOOT

Troubleshooting Matrix

Symptom → likely cause → fix. Work top-down: status, logs, network, permissions.

1 Task status stoppedReason → **2 CloudWatch logs** /ecs/<family> → **3 Network path** subnet, IP, SG → **4 IAM roles** exec role, user

Symptom	Likely cause	Fix
Browser times out on <ip>:8000	Security group lacks inbound 8000, public IP disabled, or private subnet	Add Custom TCP 8000 rule; re-run task with Public IP ENABLED in a public subnet
CannotPullContainerError	Wrong image URI/tag, missing execution role, or no route to the registry	Copy URI from ECR; attach AmazonECSTaskExecutionRolePolicy; enable public IP
exec format error in logs	Image architecture differs from task platform (ARM vs X86_64)	Rebuild on matching host or set the task runtime platform to match
Task stops, exit code 1	App crashed or listens on 127.0.0.1 / another port	Read CloudWatch logs; bind to 0.0.0.0:8000
docker: permission denied (docker.sock)	User not in the docker group	sudo usermod -aG docker $USER, then reboot / re-login
no basic auth credentials on push	Logged in to the wrong registry	Re-run login for public (us-east-1) or private (your region) ECR
Unable to locate credentials	aws configure not run, or no instance role	Run aws configure, or attach an IAM role to EC2
AccessDenied on ecr push	IAM user missing ECR policy	Attach the matching ECR PowerUser policy (Phase 3 table)
ResourceInitializationError	Task cannot reach ECR/CloudWatch over the network	Public IP + route to internet gateway, or VPC endpoints
No logs in CloudWatch	Log collection off or exec role lacks logs permission	Enable awslogs in container; ensure role has CloudWatch Logs access
port is already allocated (local)	Host port 80 in use	Use -p 8080:8000 and browse :8080

START HERE

Fastest signal: describe-tasks → stoppedReason. It usually names the exact problem in one line.

© 2026 cloudmessenger. All rights reserved.

09

BEYOND THE LAB

Production Hardening

The lab proves it works. This is what changes before real users arrive.

Area	Lab practice	Production practice	Why it matters
Registry	Public ECR repo	Private ECR, scan on push, lifecycle rules	Protects IP, finds CVEs, caps storage cost
Image tags	:latest, mutable	Immutable tags: git SHA or semver	Reproducible deploys, easy rollback
Credentials	Access keys via aws configure	IAM roles; OIDC from CI; no long-lived keys	Leaked keys are the top breach cause
Network exposure	0.0.0.0/0 on port 8000	ALB in front; task SG allows only the ALB SG	Shrinks attack surface
Task placement	Public subnet + public IP	Private subnets, NAT or VPC endpoints	No direct internet access to tasks
Run mode	Standalone task	ECS Service, 2+ tasks across AZs	Self-healing, zero-downtime deploys
Deploys	Console clicks	CI/CD pipeline with rolling deploy + circuit breaker	Repeatable, auditable, safe
Transport	HTTP on a raw IP	HTTPS via ACM certificate on the ALB	Encryption and a stable DNS name
Container hygiene	node:18, runs as root	LTS slim base, non-root USER, read-only FS	Fewer vulnerabilities
Observability	Read logs by hand	Log retention, alarms, Container Insights	Know before users do
Cost control	Remember to delete	Budgets, tags, right-sized tasks	Fargate bills while tasks run

PRODUCTION READINESS CHECKLIST

- [ ] Private ECR with image scanning on
- [ ] No access keys on servers or in Git
- [ ] CloudWatch alarms on CPU, memory, 5xx
- [ ] Execution role is least-privilege
- [ ] Task runs behind an ALB with HTTPS
- [ ] Service runs ≥ 2 tasks in 2 AZs
- [ ] Log group retention set (e.g. 30 days)
- [ ] Rollback tested with a previous tag

© 2026 cloudmessenger. All rights reserved.

10

WRAP UP

Cleanup & Cheat Sheet

Stop the bill when you are done, and keep this page handy.

CLEANUP ORDER (avoid surprise charges)

1. **Stop the task**
   ECS → Cluster → Tasks → Stop (or delete the service).
2. **Delete the cluster**
   Only after tasks are stopped.
3. **Deregister task definitions**
   Old revisions cost nothing, but tidy up.
4. **Delete ECR images / repo**
   Storage is billed per GB-month.
5. **Delete log groups**
   CloudWatch keeps logs until you remove them.
6. **Terminate the EC2 instance**
   And release any Elastic IP.
7. **Remove IAM access keys**
   Deactivate then delete keys you created for the lab.

ONE-PAGE CHEAT SHEET

docker

```
# Docker
docker build -t nodeapp .
docker run -d --name nodeappweb \
  -p 80:8000 nodeapp
docker ps
docker logs nodeappweb
docker stop nodeappweb
docker rm nodeappweb
```

aws cli

```
# AWS CLI
aws configure
aws sts get-caller-identity
aws --version
# ECR public login
aws ecr-public get-login-password \
  --region us-east-1 | docker login \
  --username AWS --password-stdin public.ecr.aws
```

ecr

```
# ECR push
docker tag node-app:latest \
  public.ecr.aws/<alias>/node-app:latest
docker push \
  public.ecr.aws/<alias>/node-app:latest
```

ecs

```
# ECS
aws ecs list-clusters
aws ecs list-tasks --cluster <c>
aws ecs describe-tasks --cluster <c> \
  --tasks <arn>
aws logs tail /ecs/<family> --follow
```

QUICK RECAP

1. Update Ubuntu, clone the repo
2. Install Docker, add user to group
3. Build and run locally on 80:8000
4. Install AWS CLI v2, aws configure
5. Create ECR repo, push image
6. Create Fargate cluster + exec role
7. Define task: image, port 8000, logs
8. Run task, open SG 8000, browse IP:8000

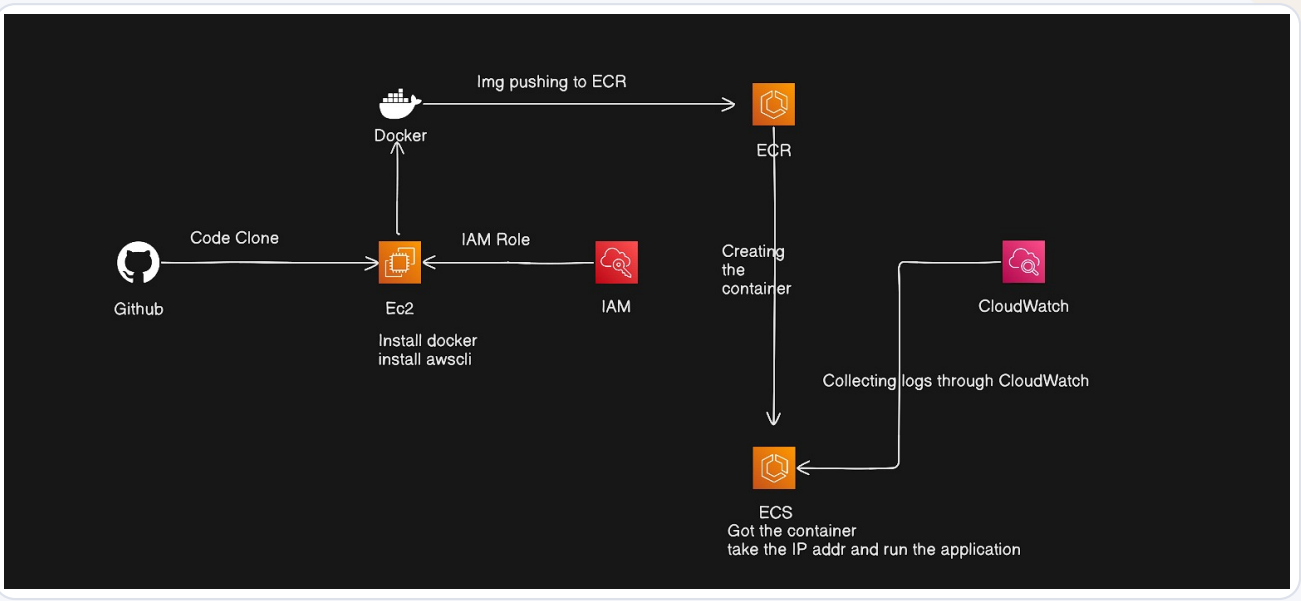
© 2026 cloudmessenger. All rights reserved.

APPENDIX · FIELD NOTES

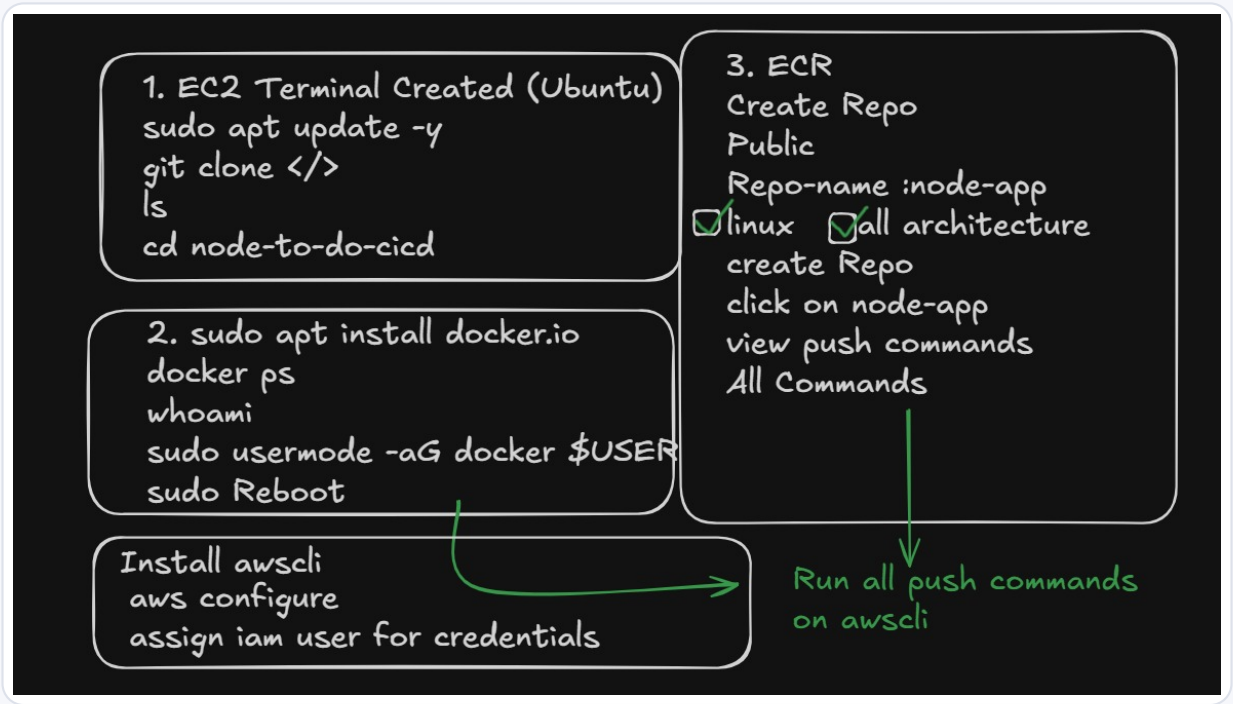
A Original Whiteboards

The workflow sketch and hand-written steps this handbook was built from.

WORKFLOW SKETCH



HAND-WRITTEN STEPS (EC2, DOCKER, ECR)



HOW TO READ

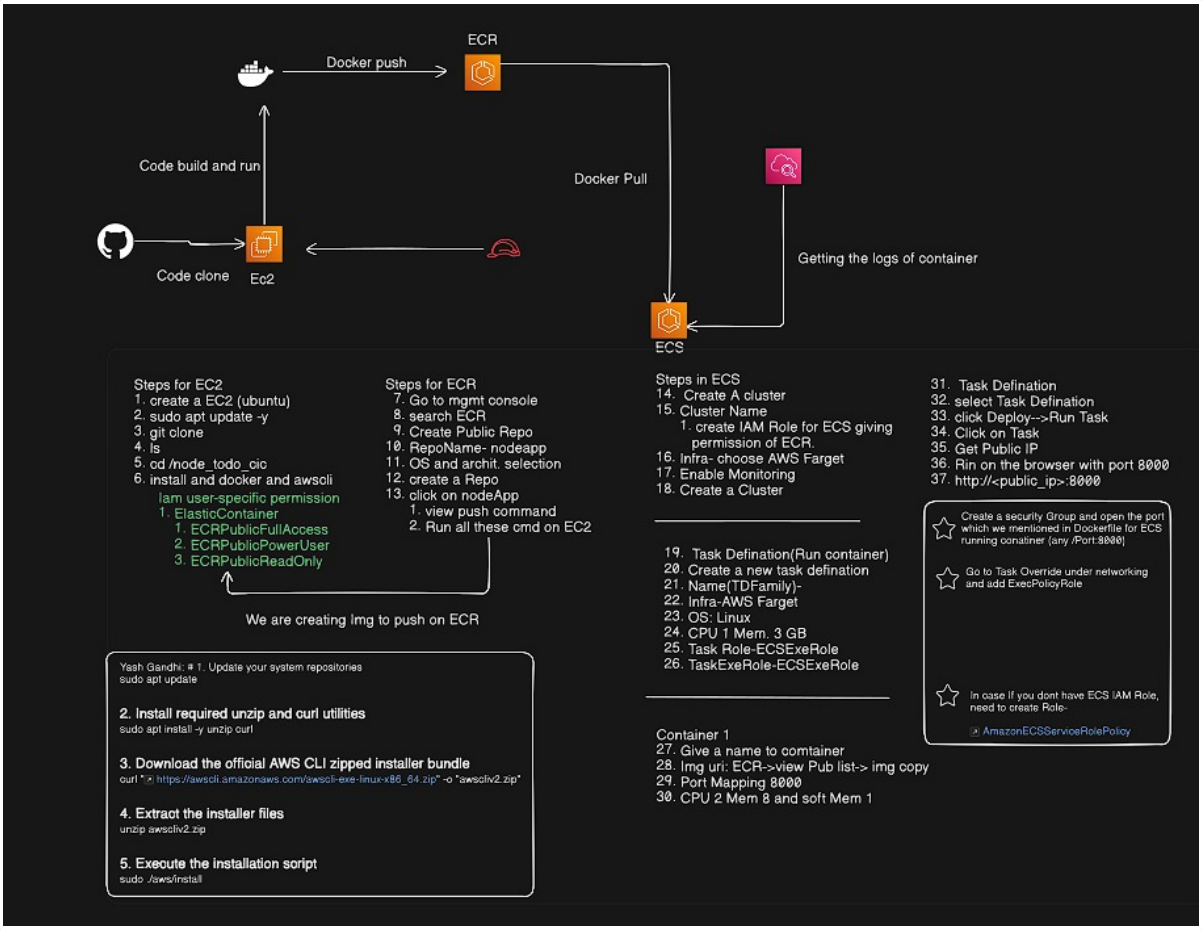
Read the sketch left to right: GitHub → EC2 (Docker + CLI, IAM role) → ECR → ECS, with CloudWatch collecting logs. It matches the architecture on page 3.

© 2026 cloudmessenger. All rights reserved.

APPENDIX · RECONCILED

Detailed Notes & Reconciliation

The full whiteboard, and where the sources disagreed.



WHERE THE ORIGINAL NOTES DISAGREED → WHAT THIS HANDBOOK DOES

Topic	In the notes	This handbook
Repository type	Public repo in whiteboards; PDF also says private	Public for the lab, private ECR for production (page 7, 12)
Architecture	ARM in the ECS infographic, “all architectures” on whiteboards	Match image and task platform; x86_64 by default
Task size	1 vCPU / 3 GB vs 0.25 vCPU / 0.5 GB	Start at 0.25 / 0.5 GB, scale on metrics
Container limits	CPU 2, memory 8 GB (above task size)	Leave empty or keep ≤ task size
Ports opened	8000 only vs 80 + 8000	8000 for ECS; 80 only for the local docker test
IAM policies	3 public-ECR policies vs 6 policies	Table with Required / Optional / Not needed (page 8)
Typos	usermode, Reboot, Defination	usermod, reboot, Definition

© 2026 cloudmessenger. All rights reserved.

Run a Node.js To-Do App with Docker



© 2026 cloudmessenger. All rights reserved.

Deploy Docker Application to AWS ECS (Fargate) via ECR

Deploy Docker Application to AWS ECS (Fargate) via ECR

Step-by-Step Guide (Based on Your Notes)

1 Create ECR Repository

Amazon ECR

1. Go to **Amazon ECR**
2. Click **Create repository**
3. Choose:
   - Visibility: Public (as per note)
   - Name: <your-repo-name>
   - Image tag mutability: Mutable
4. OS: **Linux**, **Architecture: ARM**
5. Click **Create**

2 Push Docker Image to ECR

View push commands

```
# 1. Authenticate
aws ecr get-login-password
--region <region> |
docker login --username AWS
--password-stdin
<account-id>.dkr.ecr.<region>
.amazonaws.com

# 2. Build image
docker build -t nodeapp .

# 3. Tag image
docker tag nodeapp:latest
<account-id>.dkr.ecr.<region>
.amazonaws.com/nodeapp:latest

# 4. Push image
docker push
<account-id>.dkr.ecr.<region>
.amazonaws.com/nodeapp:latest
```

3 Create ECS Cluster (Fargate)

Amazon ECS

1. Go to **Amazon ECS**
2. Click **Create cluster**
3. Enter cluster name
   - e.g. my-cluster
4. Choose **Infrastructure: AWS Fargate only**
5. Enable **Container Insights** (optional)
6. Click **Create**

4 Create Task Definition

Task Definition

1. Go to **Task definitions**
2. Click **Create new task definition**
3. Enter family name
   - e.g. nodeapp-task
4. Choose **Launch type: AWS Fargate**
5. Select **OS: Linux**
6. Select **CPU & Memory** (e.g. 0.25 vCPU, 0.5 GB)
7. **Task role**: Create/Select IAM role for ECS (if needed)
8. **Execution role**: Use same role for task execution

5 Add Container Configuration

Container settings

1. Add container
2. Container name
   - e.g. nodeapp-container
3. Image URI (from ECR)
   - e.g. <account-id>.dkr.ecr.<region>.amazonaws.com/nodeapp:latest
4. Port mapping
   - Container port : 8000
   - Protocol : TCP
5. Environment variables (if required)

6 Create Task Definition

1. Review all settings
2. Click **Create**

Task Definition created successfully!

Example: nodeapp-task:1

7 Run Task

1. Go to **Clusters** → Select your cluster
2. Click **Run new task**
3. Select **Launch type: Fargate**
4. Choose task definition (e.g. **nodeapp-task:1**)
5. Set **Desired tasks: 1**
6. Configure **Networking**:
   - VPC: (your VPC)
   - Subnet: Public subnet
   - Security group: Allow port 8000
   - Public IP: ENABLED
7. Click **Create (Run)**

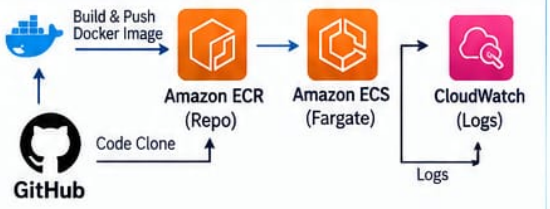
8 Access the Application

1. Go to the running task
2. Get the **Public IP** from the **Networking** section
3. Open in browser:
   - http://<public-ip>:8000
4. Check logs in **CloudWatch** (Log group: /ecs/<your-app>)

Dockerfile (Example)

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 8000
CMD ["node", "server.js"]
```

Architecture Flow



Security Group Rule

Allow inbound traffic:

Type	Protocol	Port	Source
Custom TCP	TCP	8000	0.0.0.0/0 (or your IP)

Your application is now running on ECS (Fargate)!
Access it using http://<public-ip>:8000

Important Notes:

- Your application must listen on 0.0.0.0:8000 inside the container.
- Container port in ECS should be 8000 (as per Dockerfile EXPOSE 8000).
- Use the same IAM role for task execution as mentioned.
- Ensure the security group allows port 8000.
- Check CloudWatch logs if the task fails.

Happy Deploying! 🚀
ECR → ECS → Fargate → Run Task

© 2026 cloudmessenger. All rights reserved.

Project Flow: From Local Setup to Cloud



© 2026 cloudmessenger. All rights reserved.

Ubuntu & Docker Setup

SETUP ENVIRONMENT

BUILD
LEARN
DEPLOY



Ubuntu & Docker

Get your environment ready to clone the project and run it using Docker.

Small Steps Big Builds

Ubuntu

1 Update Packages
Update your Ubuntu system packages.

```
apt update -y
```

2 Clone the Repository
Clone the project repository from GitHub.

```
git clone https://github.com/
Cloud-Messenger/node-todo-cicd.git
```

3 List Files
Verify the repository is cloned.

```
ls
```

4 Navigate to Project Directory
Move into the cloned repository directory.

```
cd node-todo-cicd
```

Docker Installation



5 Install Docker
Install Docker Engine on Ubuntu.

```
apt install docker.io
```



Environment Ready!
Ubuntu is updated, the project is cloned and Docker is installed.
You're all set for the next steps!

Let's Build Something!

CODE • CONTAIN • DEPLOY

© 2026 cloudmessenger. All rights reserved.

Install & Configure AWS CLI

AWS CLI

3 Install AWS CLI

Remove the old version (if any) and install the latest AWS CLI on Ubuntu.

Latest Tools Brighter Builds

AWS CLI

24 Remove Existing AWS CLI

Uninstall any previously installed version.

```
sudo apt remove awscli
```

25 Download AWS CLI v2

Download the latest AWS CLI installer.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

26 Unzip the Installer

Extract the downloaded file.

```
unzip awscliv2.zip
```

27 Install AWS CLI

Run the installer.

```
sudo ./aws/install
```

AWS CLI Installed!

You are now ready to configure AWS CLI.

One step closer to the cloud!

AWS CLI

4 Configure AWS CLI

Set up your AWS credentials to connect to your AWS account.

Secure Access Real Possibilities

1 Run Configure Command

Start the AWS CLI configuration.

```
aws configure
```

2 Enter Credentials

You will be prompted to enter:

- AWS Access Key ID
- AWS Secret Access Key
- Default region name (e.g., ap-south-1)
- Default output format (e.g., json)

Where to get Access Key & Secret Key?

1. Go to AWS Console → IAM → Users.
2. Select your IAM user (e.g., Tanay).
3. Go to the Security credentials tab.
4. Click **Create access key** and copy the Access Key ID and Secret Access Key.

3 Configuration Complete

Once entered, your AWS CLI will be configured and ready to use.

AWS CLI Configured!

You can now access your AWS resources from the command line.

aws | LEARN • BUILD • DEPLOY

CLOUD SKILLS BRIGHTER TOMORROW

© 2026 cloudmessenger. All rights reserved.

Create a Public ECR Repository

5

AWS ECR

Create a Public ECR Repository

BUILD
STORE
DEPLOY

Create a public repository in Amazon ECR to store and push your Docker image.

1

Open Amazon ECR

In the AWS Management Console, search for **"ECR"** and open **Amazon ECR**.



AWS Console

ECR

2

Create Public Repository

In the left navigation pane, click **Public registry → Repositories → Create repository.**

Amazon ECR

- Private registry
- Public registry
- Repositories
- Settings

3

Repository Created

Your public ECR repository is now ready.
Note down the repository URI (you'll use it to tag and push your Docker image).

Repository Ready!

Repository URI

public.ecr.aws/<alias>/tanay

Next

Your public ECR repository is ready!

You can now push your Docker image to this repository using the repository URI.

One Step Closer

CONTAINERS
ENABLE
POSSIBILITIES

aws

© 2026 cloudmessenger. All rights reserved.

Create ECS Cluster, Task & Run



© 2026 cloudmessenger. All rights reserved.

cloud-messenger™

CREATED BY

cloud-messenger™

Learn • Build • Visualize • Troubleshoot • Grow

W WEBSITE
cloudmessenger.in

L LINKEDIN
linkedin.com/in/cloud-messenger

G GITHUB
github.com/Cloud-Messenger

#buildwithCloud-Messenger

Made with care by cloud-messenger™. If this handbook helped you ship, share it with someone who is just starting out.

COPYRIGHT NOTICE

© 2026 cloudmessenger. The compilation, structure, layout, design and branding of this workbook are the original work of cloudmessenger. No part may be reproduced, redistributed or resold without written permission.